

Background on the Data Set

The original MNIST digits data set is one of the most widely used benchmark data sets in machine learning. It contains 70,000 grayscale images of handwritten digits 0 through 9, each stored as a 28 x 28 pixel image. Because it is small, clean, balanced, and easy to load, it became a standard introductory benchmark for image classification. However, that same simplicity is also a limitation: modern models often perform extremely well on MNIST, so it is no longer a very challenging benchmark. The authors of the paper argued that a new drop-in replacement for MNIST was needed for two reasons. First, they wanted a benchmark with the same convenient format as MNIST, but one that was more difficult and therefore more useful for comparing modern deep learning models. Second, they wanted a benchmark with clearer cultural and social value. Instead of classifying digits, their Kuzushiji-MNIST data set contains images of classical Japanese characters written in Kuzushiji script, a cursive style used in historical Japanese literature. The authors focused on classical Japanese characters because a very large amount of Japan's literary and historical record is effectively inaccessible to most modern readers. Kuzushiji is no longer taught in most schools, so even native Japanese speakers often cannot read texts written more than about 150 years ago. By releasing a machine learning data set in a familiar MNIST-like format, the authors hoped to attract the ML community to a meaningful cultural preservation problem. Their goal was not only to create a stronger benchmark, but also to encourage automated transcription tools that could help scholars and the public access classical Japanese documents. For this project, the data set contained 60,000 training images and 10,000 test images, all grayscale and 28 x 28 pixels, across 10 balanced classes: o, ki, su, tsu, na, ha, ma, ya, re, and wo. Because each image belongs to exactly one of ten categories, this is a multiclass classification problem. The most appropriate Keras loss function was therefore `sparse_categorical_crossentropy`, with a final dense layer of 10 softmax outputs.

Summaries of the Optimized DNN and CNN Models

Optimized DNN (MLP)

The notebook selected the following DNN as the best validation model:

- Configuration: `mlp_h2_u64_32_rmsprop_e15`
- Validation accuracy: 0.9448
- Test accuracy: 0.8618

Architecture summary:

- Input: 28 x 28 x 1
- Flatten: output size 784
- Dense layer: 64 units, ReLU
- Dense layer: 32 units, ReLU
- Dense output layer: 10 units, softmax
- Trainable parameters: 52,650

Parameter calculations:

- Flatten layer: 0 parameters
- First dense layer: $784 \times 64 + 64 = 50,240$
- Second dense layer: $64 \times 32 + 32 = 2,080$
- Output layer: $32 \times 10 + 10 = 330$

Total trainable parameters:

- $50,240 + 2,080 + 330 = 52,650$

Optimized CNN

The notebook selected the following CNN as the best validation model:

- Configuration: cnn_c32_64_k5_pool_adam_e15
- Validation accuracy: 0.9843
- Test accuracy: 0.9471

Architecture summary:

- Input: 28 x 28 x 1
- Conv2D: 32 filters, 5 x 5, ReLU
- Conv2D: 64 filters, 5 x 5, ReLU
- MaxPooling2D: 2 x 2
- Flatten
- Dense layer: 64 units, ReLU
- Dense layer: 32 units, ReLU

- Dense output layer: 10 units, softmax
- Trainable parameters: 857,386

Parameter calculations:

- First convolution layer: $(5 \times 5 \times 1 + 1) \times 32 = 832$
- Second convolution layer: $(5 \times 5 \times 32 + 1) \times 64 = 51,264$
- Max pooling layer: 0 parameters
- After pooling, feature map size becomes $14 \times 14 \times 64 = 12,544$
- First dense layer: $12,544 \times 64 + 64 = 802,880$
- Second dense layer: $64 \times 32 + 32 = 2,080$
- Output layer: $32 \times 10 + 10 = 330$

Total trainable parameters:

- $832 + 51,264 + 802,880 + 2,080 + 330 = 857,386$

A useful detail from the Keras summaries is that they also show larger “total params” values because Keras counts optimizer state separately. For explaining model complexity, the most relevant number is the trainable parameter count.

How Hyper-Parameters Changed the Number of Parameters

For dense layers, the parameter count grows according to:

- inputs x outputs + outputs

This means increasing the number of hidden nodes can increase the parameter count very quickly, especially in the first dense layer, because it receives all 784 flattened pixels. For example, moving from 32 to 64 nodes in the first hidden layer substantially increased the DNN’s parameter count. For convolution layers, the formula is:

- $(\text{kernel height} \times \text{kernel width} \times \text{input channels} + 1) \times \text{number of filters}$

This means more filters and larger kernels both increase parameters, but usually not as dramatically as a large dense layer. In the CNN, the biggest parameter jump did not come from the convolution layers themselves, but from the first dense layer after flattening. Max pooling reduced the spatial size from 28×28 to 14×14 , which greatly reduced the size of the flattened vector and therefore greatly reduced the number of dense-layer parameters. This is clear when comparing pooled and non-pooled CNNs.

The 32/64, k=5 CNN without pooling had 3,265,834 trainable parameters, while the pooled version had only 857,386. Pooling removed more than two million parameters and still improved accuracy.

Impact of the Various Parameters on Model Performance

The MLP experiments showed that width mattered more than depth at the beginning. Very small hidden layers, especially the 8/8 models, performed poorly even when more layers were added. The small models mostly stayed around the high-60% to low-70% range on test accuracy, which means that simply stacking more narrow dense layers was not enough. In contrast, increasing the hidden-layer widths to 32/32 or 64/32 produced a large jump in performance. The best MLP validation result came from the 64/32 configuration with two hidden layers and RMSprop for 15 epochs. The number of dense layers helped somewhat, but only after the network had enough units to represent the data well. Moving from zero hidden layers to one or two reasonably wide hidden layers helped a lot. After that, the benefit of adding a third dense layer was small. In fact, a nearby three-hidden-layer model slightly outperformed the chosen MLP on test accuracy, but the notebook correctly selected models by validation accuracy rather than by test accuracy. That suggests that once the model was already reasonably wide, extra depth gave diminishing returns. The optimizer had a smaller effect than architecture. In the MLP runs, RMSprop often edged out Adam on the strongest configurations, but the difference was not dramatic. The main gains came from using larger hidden layers, not from changing the optimizer. The CNN experiments showed a much clearer pattern. Increasing the convolution capacity from 16/32 filters to 32/64 filters improved performance. Increasing kernel size from 3 x 3 to 5 x 5 also helped, especially in the stronger 32/64 models. The largest and most useful structural change was adding max pooling. Pooling both reduced parameters substantially and improved or preserved accuracy in most of the strongest CNN settings. That makes pooling one of the best time-efficient design choices in this project. Epoch count helped, but not consistently. In some cases 15 epochs improved validation accuracy slightly, but in other cases the best test result appeared at 10 epochs. For example, the selected best CNN by validation accuracy used 15 epochs, but the highest CNN test accuracy in the grid actually came from the very similar `cnn_c32_64_k5_pool_adam_e10` model at 0.9527. This suggests that once a good CNN architecture is found, very long extra training may offer only small gains and may sometimes slightly overfit. If I were advising someone doing a similar project, I would suggest this order of effort:

- First, make sure the model has enough capacity.

- For an MLP, increase hidden-layer width before adding more layers.
- For a CNN, use convolution first, then try pooling, then increase filters or kernel size.
- Spend less time obsessing over optimizer choice unless the architecture is already strong.
- After a good architecture is found, check whether a smaller number of epochs already gives nearly the same result.

Which Model Performed Better, and Why?

The CNN performed much better than the DNN. The optimized DNN achieved:

- Validation accuracy: 94.48%
- Test accuracy: 86.18%

The optimized CNN achieved:

- Validation accuracy: 98.43%
- Test accuracy: 94.71%

So the CNN outperformed the DNN by about 8.5 percentage points on the test set. This result makes sense because the data are images.

A fully connected DNN flattens the image into a one-dimensional vector, which throws away most of the spatial structure. Neighboring pixels are no longer treated as neighbors in a meaningful visual way. A CNN preserves local spatial patterns through convolution, so it can learn edges, curves, stroke fragments, and larger stroke combinations that are especially important in handwritten character recognition. This matters even more for Kuzushiji characters, where subtle local stroke differences can separate one class from another. Another reason the CNN likely performed better is that the classes include multiple writing variations of the same character. Convolutional layers are better at recognizing these local visual patterns even when they appear in slightly different positions or forms. The DNN had to learn these relationships only through dense connections, which is less efficient for images.

Interpretation of the Confusion Matrices

The DNN's errors were not balanced across classes. Its per-class accuracies ranged from only 75.0% for su to 93.1% for ma. The weakest DNN classes were:

- su: 75.0%

- ya: 80.2%
- na: 83.7%
- ki: 84.7%

Its strongest classes were:

- ma: 93.1%
- tsu: 91.1%
- o: 90.9%

This spread shows that the MLP struggled much more with some characters than others. The largest DNN confusions included su -> ki, su -> tsu, ya -> ma, ki -> ma, and na -> o. These are not random mistakes. They suggest that some classes share similar stroke shapes or global silhouettes when reduced to 28 x 28 grayscale images. Since the DNN does not model local structure well, it was more vulnerable to these similarities. The CNN's confusion matrix was much better balanced. Its per-class accuracies ranged from 89.6% for su to 98.8% for re. The weakest CNN classes were:

- su: 89.6%
- ki: 91.6%
- na: 91.7%

Its strongest classes were:

- re: 98.8%
- ma: 98.1%
- tsu: 96.7%
- ya: 96.7%

The CNN still made more mistakes on some classes than others, but the differences were much smaller than in the DNN. That means the CNN learned more robust class-specific visual features. Some confusions remained across both models. One of the most persistent was ki -> ma, which was the largest remaining CNN off-diagonal error and also a large error for the DNN. The CNN also still confused su -> tsu, su -> ma, and na -> re more than other class pairs.

These patterns suggest that those classes likely share visually

similar brushstroke structures or multiple variant forms. Because Kuzushiji includes hentaigana, one class can have several visually different writing styles, which increases within-class variation and makes some distinctions harder. Overall, the DNN confusion matrix suggests uneven learning, with a few classes learned well and a few learned poorly. The CNN confusion matrix suggests much more consistent learning across all classes, although a small set of visually similar pairs remained difficult.

If this project were extended, the next useful step would be to display misclassified examples next to correctly classified examples from the true and predicted classes. That would help confirm whether the remaining errors are caused by shared strokes, ambiguous shapes, or especially unusual writing variants.

Conclusion

This project showed that Kuzushiji-MNIST is a more challenging and more meaningful benchmark than the original MNIST digits data set. The best DNN achieved solid performance, but the CNN was clearly superior because it could exploit the spatial structure of the images. The strongest improvements came from using a convolutional architecture, increasing convolution strength to 32/64 filters, using 5 x 5 kernels, and including max pooling. In contrast, simply adding more narrow dense layers to the MLP did not help much. The results support the main idea of the paper: a culturally meaningful data set can also be a useful machine learning benchmark. In this case, the best model was not just more accurate in general, but also more balanced across classes, which is especially important for real transcription tasks involving historical writing. If you want, I can turn this into a more polished “lab report” version with a title page, introduction, results, discussion, and conclusion format for direct pasting into Word.